1 mxcli: AI-Powered Development for Mendix

Accelerate Mendix Development with Natural Language and Code Generation

1.1 Executive Summary

mxcli is a command-line interface that brings AI-powered development to the Mendix platform. By providing a SQL-like language called **MDL (Mendix Definition Language)**, mxcli enables developers and AI assistants to read, understand, and modify Mendix projects programmatically.

1.1.1 Key Benefits

- **10x Faster Development**: Generate domain models, microflows, and pages using natural language
- **Consistent Standards**: Define organizational best practices as MDL templates
- **AI-Native Integration**: Purpose-built for GenAI code assistants like Claude Code
- **Full Project Understanding**: Agentic search capabilities for intelligent code navigation
- **Token-Efficient**: MDL uses 5-10x fewer tokens than JSON model representations, reducing API costs
- **Better Code Reviews**: Higher information density makes diffs easier to read and review

---

1.2 How It Works

```
┌──────────────────┐      ┌────────────────┐      ┌──────────────────┐
│   Developer      │      │    mxcli       │      │ Mendix Project   │
│   or AI Agent    │──────►│  (MDL REPL)    │──────►│   (.mpr file)    │
│                  │ MDL  │                │      │                  │
│ "Create a        │      │ Parses MDL     │      │ Creates actual   │
│  Customer        │      │ Validates      │      │ entities,        │
│  entity..."      │      │ Executes       │      │ microflows,      │
└──────────────────┘      └────────────────┘      │ pages, etc.      │
                                                  └──────────────────┘
```

1.2.1 The MDL Advantage

MDL is a **text-based, SQL-like language** that describes Mendix model elements. Compared to the visual Studio Pro interface and JSON model representations, MDL offers:

- **Generated by AI**: LLMs produce syntactically correct MDL with high accuracy
- **Human-Readable Diffs**: Text-based format is easier to review than binary or JSON changes
- **High Information Density**: Scan complete microflow logic at a glance, without clicking through activities
- **Token Efficient**: 5-10x fewer tokens than JSON representations, allowing more application context in LLM prompts
- **Concise Skill Documents**: Smaller, more precise AI instruction files lead to better generation quality
- **Templated for Reuse**: Standard patterns become simple copy-paste snippets

1.2.2 Why Token Efficiency Matters

When working with GenAI APIs, **context window size is a critical constraint**. MDL's compact syntax provides significant advantages over JSON model representations:

Representation	Tokens for a 10-entity module
JSON (raw model)	~15,000-25,000 tokens
MDL	~2,000-4,000 tokens
Savings	**5-10x fewer tokens**

Benefits of token efficiency:

1. **Lower API costs**: Fewer tokens = lower costs per request
2. **More context available**: Fit more of your application in a single prompt
3. **Better AI understanding**: Less noise, more signal in the context
4. **Faster responses**: Smaller payloads process more quickly
5. **Larger scope operations**: Modify multiple entities, microflows, and pages in one request

Example comparison:

```
// JSON representation (~180 tokens)
{
  "entity": {
    "name": "Customer",
    "documentation": "Customer master data",
    "attributes": [
      {"name": "Name", "type": {"type": "String", "length": 200}},
      {"name": "Email", "type": {"type": "String", "length": 200}}
    ]
  }
}
```

```
-- MDL representation (~35 tokens)
/** Customer master data */
CREATE PERSISTENT ENTITY Sales.Customer (
    Name: String(200),
    Email: String(200)
);
```

This efficiency extends to skill documents—the AI instruction files that guide generation. Concise MDL examples in skills mean: - More patterns fit in the skill document - AI has clearer examples to follow - Generated output is more accurate and consistent

---

1.3 Integration with GenAI Tools

1.3.1 Multi-Tool AI Assistant Support

mxcli is designed from the ground up to work with AI coding assistants. It supports **five AI tools** out of the box:

Tool	Flag	What's Created
Claude Code (default)	`--tool claude`	`.claude/settings.json`, `CLAUDE.md`, commands, lint rules
Cursor	`--tool cursor`	`.cursorrules`
Continue.dev	`--tool continue`	`.continue/config.json`
Windsurf	`--tool windsurf`	`.windsurfrules`
Aider	`--tool aider`	`.aider.conf.yml`

```
# Initialize with Claude Code (default)
mxcli init

# Initialize for Cursor
mxcli init --tool cursor

# Initialize for multiple tools
mxcli init --tool claude --tool cursor --tool continue

# Initialize for all supported tools
mxcli init --all-tools

# List supported tools
mxcli init --list-tools
```

When you run `mxcli init`, it creates a **universal** structure shared by all tools, plus tool-specific configuration:

```
your-mendix-project/
├── .ai-context/
│   ├── skills/               # MDL pattern guides (shared by all tools)
│   │   ├── write-microflows.md
│   │   ├── create-page.md
│   │   ├── overview-pages.md
│   │   └── generate-domain-model.md
│   └── examples/             # MDL example scripts
├── PROJECT_AI.md             # Universal AI assistant guide
├── .claude/                  # Claude Code specific (if selected)
│   ├── settings.json
│   ├── commands/             # Slash commands
│   └── lint-rules/           # Starlark lint rules
├── CLAUDE.md                 # Claude Code project context
├── .cursorrules              # Cursor rules (if selected)
├── .windsurfrules            # Windsurf rules (if selected)
├── .continue/config.json     # Continue.dev config (if selected)
├── .aider.conf.yml           # Aider config (if selected)
└── mxcli                     # CLI executable
```

The AI assistant can then:

1. **Understand your project** by querying the model
2. **Generate MDL** based on natural language requests
3. **Validate changes** before applying them

4. **Learn your patterns** from existing model elements

1.3.2 Example: Natural Language to Working Feature

Developer says: *"Create a customer management module with name, email, and phone fields. Add a microflow to validate the email format."*

Claude Code generates:

```
/** Create the customer management module */
CREATE MODULE CustomerManagement;

/**
 * Customer entity with contact information.
 * Master data for CRM functionality.
 */
@Position(100, 100)
CREATE PERSISTENT ENTITY CustomerManagement.Customer (
    Name: String(200) NOT NULL,
    Email: String(200) NOT NULL UNIQUE,
    Phone: String(50),
    IsActive: Boolean DEFAULT true,
    CreatedAt: DateTime DEFAULT '[%CurrentDateTime%]'
);

/**
 * Validate email format.
 * Returns true if email contains @ and . characters.
 */
CREATE MICROFLOW CustomerManagement.ValidateEmail(
    $email: String
) RETURNS Boolean AS $isValid
BEGIN
    DECLARE $isValid Boolean = false;

    IF $email != empty AND contains($email, '@') AND contains($email, '.') THEN
        SET $isValid = true;
    END IF;

    RETURN $isValid;
END;
```

Result: A fully functional module appears in Studio Pro, ready for use.

1.4 MDL by Example

Note: All examples use **V3 syntax** - the current MDL syntax for pages and widgets. V3 uses: - Curly braces `{ }` for widget nesting (not `BEGIN/END`) - Property syntax with colons: `(Title: 'value', Layout: 'value')` - Explicit property names: `Label:`, `Binds:`, `DataSource:`, `Action:`

For complete V3 syntax reference, see the skills documentation.

1.4.1 Modules and Entities

The `@Position(x, y)` annotation controls where entities appear in the Studio Pro domain model diagram. Coordinates are in pixels from the top-left. This helps maintain clean, organized visual layouts when generating multiple entities.

```
/** Create a new module for sales functionality */
CREATE MODULE Sales;

/**
 * Order entity with business rules and audit support.
 * Sales orders with full audit trail.
 */
@Position(100, 100)
CREATE PERSISTENT ENTITY Sales.Order (
    /** Unique order identifier for external systems */
    OrderNumber: String(50) NOT NULL UNIQUE,

    /** Order timestamp, defaults to creation time */
    OrderDate: DateTime DEFAULT '[%CurrentDateTime%]',

    /** Monetary value with precision */
    TotalAmount: Decimal NOT NULL DEFAULT 0,

    /** Order lifecycle status */
    Status: Enum Sales.OrderStatus DEFAULT 'Draft',

    /** Soft delete support */
    IsArchived: Boolean DEFAULT false
);

/**
 * Order status enumeration for lifecycle tracking.
 * Lifecycle states for sales orders.
 */
CREATE ENUMERATION Sales.OrderStatus (
    Draft CAPTION 'Draft Order',
    Submitted CAPTION 'Submitted for Processing',
    Approved CAPTION 'Approved',
    Shipped CAPTION 'Shipped to Customer',
    Completed CAPTION 'Order Completed',
    Cancelled CAPTION 'Order Cancelled'
);

/**
 * Link orders to customers.
 * Each order belongs to exactly one customer.
 */
CREATE ASSOCIATION Sales.Order_Customer
    FROM Sales.Order
    TO Sales.Customer
    TYPE REFERENCE
    OWNER DEFAULT
    DELETE_BEHAVIOR DELETE_BUT_KEEP_REFERENCES;
```

1.4.2 Microflows (Business Logic)

```
/**
 * Process a new order submission
 * Validates the order, updates inventory, and sends confirmation
 */
CREATE MICROFLOW Sales.SubmitOrder(
    $order: Sales.Order
) RETURNS Boolean AS $success
FOLDER 'Order Processing'
BEGIN
    DECLARE $success Boolean = false;

    /** Validate order has items */
    RETRIEVE $orderLines FROM Sales.OrderLine
        WHERE Sales.OrderLine_Order = $order;

    IF length($orderLines) = 0 THEN
        LOG WARNING NODE 'OrderValidation' 'Order has no items, cannot submit';
        RETURN false;
    END IF;

    /** Calculate total from line items */
    DECLARE $total Decimal = 0;
    LOOP $line IN $orderLines
    BEGIN
        SET $total = $total + $line/LineTotal;
    END LOOP;

    /** Update order status and commit */
    CHANGE $order (
        TotalAmount = $total,
        Status = 'Submitted',
        SubmittedAt = [%CurrentDateTime%]);
    COMMIT $order WITH EVENTS;

    /** Send confirmation email */
    CALL MICROFLOW Sales.SendOrderConfirmation(order = $order);

    SET $success = true;
    RETURN $success;
END;

/**
 * Retrieve orders for dashboard display
 * Returns recent orders with customer information
 */
CREATE MICROFLOW Sales.GetRecentOrders()
RETURNS List of Sales.Order AS $orders
BEGIN
    RETRIEVE $orders FROM (
        SELECT o.*
        FROM Sales.Order AS o
        WHERE o/IsArchived = false
```

```
        ORDER BY o/OrderDate DESC
        LIMIT 50
    );

    RETURN $orders;
END;
```

1.4.3 Pages (User Interface)

```
/**
 * Customer overview page with data grid
 * Allows viewing, filtering, and managing customers
 */
CREATE PAGE Sales.CustomerOverview
(
  Title: 'Customer Management',
  Layout: Atlas_Core.Atlas_Default,
  Url: 'customer-overview'
)
{
  LAYOUTGRID mainGrid {
    ROW row1 {
      COLUMN col1 (DesktopWidth: 12) {
        DYNAMICTEXT pageTitle (Content: 'Customer Overview', RenderMode: H3)
      }
    }
    ROW row2 {
      COLUMN col2 (DesktopWidth: 12) {
        DATAGRID CustomerGrid (
          DataSource: DATABASE Sales.Customer,
          OrderBy: Name ASC
        ) {
          COLUMN colName (Attr: Name, Caption: 'Customer Name')
          COLUMN colEmail (Attr: Email, Caption: 'Email Address')
          COLUMN colPhone (Attr: Phone, Caption: 'Phone')
          COLUMN colActive (Attr: IsActive, Caption: 'Active')
          COLUMN colCreated (Attr: CreatedAt, Caption: 'Created')
        }
      }
    }
  }
}

/**
 * Customer edit form
 * Used for both new and existing customers
 */
CREATE PAGE Sales.Customer_NewEdit
(
  Params: { $Customer: Sales.Customer },
  Title: 'Customer Details',
  Layout: Atlas_Core.PopupLayout
```

```
)
{
  DATAVIEW dvCustomer (DataSource: $Customer) {
    TEXTBOX txtName (Label: 'Full Name', Binds: Name)
    TEXTBOX txtEmail (Label: 'Email Address', Binds: Email)
    TEXTBOX txtPhone (Label: 'Phone Number', Binds: Phone)
    CHECKBOX cbActive (Label: 'Active Customer', Binds: IsActive)

    FOOTER footer1 {
      ACTIONBUTTON btnSave (Caption: 'Save', Action: SAVE_CHANGES, ButtonStyle:
Success)
      ACTIONBUTTON btnCancel (Caption: 'Cancel', Action: CANCEL_CHANGES)
    }
  }
}
```

1.4.4 Layout Grids (Responsive Design)

```
/**
 * Dashboard page with responsive layout grid
 * Uses 12-column grid system with breakpoints
 */
CREATE PAGE Sales.Dashboard
(
  Title: 'Sales Dashboard',
  Layout: Atlas_Core.Atlas_Default,
  Url: 'dashboard'
)
{
  LAYOUTGRID dashboardGrid {
    /** Page header row */
    ROW headerRow {
      COLUMN col1 (DesktopWidth: 12) {
        DYNAMICTEXT pageTitle (Content: 'Sales Dashboard', RenderMode: H2)
        DYNAMICTEXT welcomeText (Content: 'Welcome to your dashboard')
      }
    }

    /** KPI cards row - responsive 3-column layout */
    ROW kpiRow {
      COLUMN colKpi1 (DesktopWidth: 4) {
        LAYOUTGRID kpiCard1 {
          ROW r1 {
            COLUMN c1 (DesktopWidth: 12) {
              DYNAMICTEXT kpi1Title (Content: 'Total Orders', RenderMode: H4)
              DYNAMICTEXT kpi1Value (Content: 'Total Orders', RenderMode: H2)
            }
          }
        }
      }
      COLUMN colKpi2 (DesktopWidth: 4) {
        LAYOUTGRID kpiCard2 {
```

```
          ROW r2 {
            COLUMN c2 (DesktopWidth: 12) {
              DYNAMICTEXT kpi2Title (Content: 'Revenue This Month', RenderMode: H4)
              DYNAMICTEXT kpi2Value (Content: 'Monthly Revenue', RenderMode: H2)
            }
          }
        }
      }
      COLUMN colKpi3 (DesktopWidth: 4) {
        LAYOUTGRID kpiCard3 {
          ROW r3 {
            COLUMN c3 (DesktopWidth: 12) {
              DYNAMICTEXT kpi3Title (Content: 'Active Customers', RenderMode: H4)
              DYNAMICTEXT kpi3Value (Content: 'Active Customers', RenderMode: H2)
            }
          }
        }
      }
    }

    /** Content row - recent orders */
    ROW contentRow {
      COLUMN colMain (DesktopWidth: 8) {
        DYNAMICTEXT chartTitle (Content: 'Monthly Sales Trend', RenderMode: H4)
      }
      COLUMN colSide (DesktopWidth: 4) {
        DYNAMICTEXT recentTitle (Content: 'Recent Orders', RenderMode: H4)
        GALLERY recentOrders (
          DataSource: MICROFLOW Sales.GetRecentOrders
        ) {
          DYNAMICTEXT orderNum (Content: 'Order', RenderMode: H4)
          DYNAMICTEXT orderDate (Content: 'Date')
        }
      }
    }
  }
}

/**
 * Nested layout grid example
 * Demonstrates nesting grids for complex layouts
 */
CREATE PAGE Sales.NestedLayoutExample
(
  Title: 'Nested Layout Demo',
  Layout: Atlas_Core.Atlas_Default,
  Url: 'nested-layout'
)
{
  LAYOUTGRID outerGrid {
    ROW row1 {
      COLUMN col1 (DesktopWidth: AutoFill) {
        LAYOUTGRID innerGrid {
          ROW innerRow1 {
```

```
            COLUMN ic1 (DesktopWidth: AutoFill) {
              DYNAMICTEXT text1 (Content: 'Column 1', RenderMode: H4)
            }
          }
          ROW innerRow2 {
            COLUMN ic2 (DesktopWidth: AutoFill) {
              DYNAMICTEXT text2 (Content: 'Column 2', RenderMode: H4)
            }
            COLUMN ic3 (DesktopWidth: AutoFill) {
              DYNAMICTEXT text3 (Content: 'Column 3', RenderMode: H4)
            }
          }
        }
      }
    }
    ROW row2 {
      COLUMN colSave (DesktopWidth: AutoFill) {
        ACTIONBUTTON btnSave (Caption: 'Save', Action: SAVE_CHANGES, ButtonStyle:
Primary)
      }
      COLUMN colCancel (DesktopWidth: AutoFill) {
        ACTIONBUTTON btnCancel (Caption: 'Cancel', Action: CANCEL_CHANGES)
      }
    }
  }
}
```

1.4.5 Snippets (Reusable UI Components)

```
/**
 * Navigation menu snippet for overview pages
 * Provides consistent navigation across the application
 */
CREATE SNIPPET Common.NavigationMenu
{
  LAYOUTGRID navGrid {
    ROW row1 {
      COLUMN col1 (DesktopWidth: 12) {
        ACTIONBUTTON btnProducts (Caption: 'Products', Action: SHOW_PAGE
Sales.ProductOverview)
        ACTIONBUTTON btnCustomers (Caption: 'Customers', Action: SHOW_PAGE
Sales.CustomerOverview)
        ACTIONBUTTON btnOrders (Caption: 'Orders', Action: SHOW_PAGE
Sales.OrderOverview)
      }
    }
  }
}

/**
 * Reusable address form snippet with parameter
 * Can be embedded in any page that needs address input
```

```
 *
 * @param $address The address entity to edit
 */
CREATE SNIPPET Common.AddressForm
(
  Params: { $address: Common.Address }
)
{
  DATAVIEW addressView (DataSource: $address) {
    TEXTBOX txtStreet (Label: 'Street Address', Binds: Street)
    TEXTBOX txtCity (Label: 'City', Binds: City)
    TEXTBOX txtState (Label: 'State/Province', Binds: State)
    TEXTBOX txtPostalCode (Label: 'Postal Code', Binds: PostalCode)
    COMBOBOX cmbCountry (Label: 'Country', Binds: Country)
  }
}

/**
 * Reusable audit info snippet
 * Shows created/modified timestamps and users
 *
 * @param $entity The auditable entity
 */
CREATE SNIPPET Common.AuditInfo
(
  Params: { $entity: System.AuditableEntity }
)
{
  LAYOUTGRID auditGrid {
    ROW row1 {
      COLUMN colCreated (DesktopWidth: 6) {
        DYNAMICTEXT createdLabel (Content: 'Created:')
        DYNAMICTEXT createdDate (Content: 'Date')
        DYNAMICTEXT createdBy (Content: 'By')
      }
      COLUMN colModified (DesktopWidth: 6) {
        DYNAMICTEXT modifiedLabel (Content: 'Modified:')
        DYNAMICTEXT modifiedDate (Content: 'Date')
        DYNAMICTEXT modifiedBy (Content: 'By')
      }
    }
  }
}

/**
 * Dashboard header snippet
 */
CREATE SNIPPET Common.DashboardHeader
{
  LAYOUTGRID headerGrid {
    ROW row1 {
      COLUMN col1 (DesktopWidth: 12) {
        DYNAMICTEXT dashTitle (Content: 'Dashboard', RenderMode: H2)
        DYNAMICTEXT dashSubtitle (Content: 'Welcome to your dashboard')
```

```
      }
    }
  }
}

/**
 * Using snippets in a page
 *
 * @param $customer The customer to display
 */
CREATE PAGE Sales.CustomerDetails
(
  Params: { $customer: Sales.Customer },
  Title: 'Customer Details',
  Layout: Atlas_Core.Atlas_Default,
  Url: 'customer-details'
)
{
  LAYOUTGRID mainGrid {
    ROW navRow {
      COLUMN col1 (DesktopWidth: 12) {
        /** Navigation menu snippet (no parameters) */
        SNIPPETCALL navMenu (Snippet: Common.NavigationMenu)
      }
    }
    ROW titleRow {
      COLUMN col2 (DesktopWidth: 12) {
        DYNAMICTEXT pageTitle (Content: 'Customer Information', RenderMode: H3)
      }
    }
    ROW contentRow {
      COLUMN colMain (DesktopWidth: 8) {
        DATAVIEW customerView (DataSource: $customer) {
          TEXTBOX txtName (Label: 'Customer Name', Binds: Name)
          TEXTBOX txtEmail (Label: 'Email', Binds: Email)
          TEXTBOX txtPhone (Label: 'Phone', Binds: Phone)

          /** Embed the reusable address snippet with parameter */
          SNIPPETCALL addressForm (Snippet: Common.AddressForm)

          FOOTER footer1 {
            ACTIONBUTTON btnSave (Caption: 'Save', Action: SAVE_CHANGES, ButtonStyle:
Success)
            ACTIONBUTTON btnCancel (Caption: 'Cancel', Action: CANCEL_CHANGES)
          }
        }
      }
      COLUMN colSide (DesktopWidth: 4) {
        /** Embed audit info snippet */
        SNIPPETCALL auditInfo (Snippet: Common.AuditInfo)
      }
    }
  }
}
```

1.4.6 Security Management

```
/**
 * Module roles define permissions within a module.
 * Create roles for different access levels.
 */
CREATE MODULE ROLE Shop.Admin DESCRIPTION 'Full administrative access';
CREATE MODULE ROLE Shop.User DESCRIPTION 'Regular user access';
CREATE MODULE ROLE Shop.Viewer DESCRIPTION 'Read-only access';

/**
 * Grant microflow execute access to roles.
 */
GRANT EXECUTE ON MICROFLOW Shop.ACT_Customer_Create TO Shop.User, Shop.Admin;
GRANT EXECUTE ON MICROFLOW Shop.ACT_Customer_Delete TO Shop.Admin;

/**
 * Grant page view access to roles.
 */
GRANT VIEW ON PAGE Shop.Customer_Overview TO Shop.User, Shop.Admin, Shop.Viewer;
GRANT VIEW ON PAGE Shop.Customer_Edit TO Shop.User, Shop.Admin;

/**
 * Entity access rules with CRUD permissions.
 * Supports selective member access and XPath constraints.
 */
GRANT Shop.Admin ON Shop.Customer (CREATE, DELETE, READ *, WRITE *);
GRANT Shop.User ON Shop.Customer (READ (Name, Email), WRITE (Email));
GRANT Shop.Viewer ON Shop.Customer (READ *);
GRANT Shop.User ON Shop.Order (READ *, WRITE *) WHERE '[Status = ''Open'']';

/**
 * User roles aggregate module roles from multiple modules.
 */
CREATE USER ROLE AppAdmin (Shop.Admin) MANAGE ALL ROLES;
CREATE USER ROLE AppUser (Shop.User);

/**
 * Project security settings.
 */
ALTER PROJECT SECURITY LEVEL PRODUCTION;
ALTER PROJECT SECURITY DEMO USERS ON;

/**
 * Demo users for development/testing.
 */
CREATE DEMO USER 'demo_admin' PASSWORD 'Admin123!' (AppAdmin);

/**
 * Inspect security with SHOW commands.
 */
SHOW PROJECT SECURITY;
SHOW MODULE ROLES IN Shop;
```

```
SHOW ACCESS ON MICROFLOW Shop.ACT_Customer_Create;
SHOW SECURITY MATRIX IN Shop;
```

1.4.7 Widget Styling

MDL supports inline styling and design properties on widgets, plus commands for discovering and modifying styling on existing pages:

```
/** Inline styling in CREATE PAGE */
CREATE PAGE MyModule.StyledPage
(
  Title: 'Styled Page',
  Layout: Atlas_Core.Atlas_Default
)
{
  CONTAINER ctn1 (
    Class: 'card mx-spacing-top-large',
    Style: 'padding: 16px;',
    DesignProperties: ['Spacing top': 'Large', 'Full width': ON]
  ) {
    DYNAMICTEXT txt1 (Content: 'Styled content', RenderMode: H3)
  }
}

/** Discover available design properties from theme */
SHOW DESIGN PROPERTIES;
SHOW DESIGN PROPERTIES FOR CONTAINER;

/** Inspect styling on existing pages */
DESCRIBE STYLING ON PAGE MyModule.CustomerEdit;
DESCRIBE STYLING ON PAGE MyModule.CustomerEdit WIDGET btnSave;

/** Modify styling on existing pages */
ALTER STYLING ON PAGE MyModule.CustomerEdit WIDGET btnSave SET Class 'btn-lg';
ALTER STYLING ON PAGE MyModule.CustomerEdit WIDGET ctn1 SET 'Spacing top' 'Large';
ALTER STYLING ON SNIPPET MyModule.NavMenu WIDGET ctn1 SET Class 'sidebar';
```

1.5 AI Generation Skills: Combining Capabilities

mxcli includes specialized **skills** that guide AI assistants in generating complete, production-ready features. These skills can be combined to create entire application modules from a single request.

1.5.1 Available Generation Skills

Skill	Purpose	Output
Domain Model	Generate entities with relationships	Entities, associations, enumerations
Overview Pages	Create list/grid pages for entities	Data grids with search, sort, CRUD buttons
Detail Pages	Generate edit/view forms	Data views with input widgets
Validation Microflows	Add business rule validation	Microflows with feedback messages
CRUD Microflows	Generate create/update/delete logic	Standard data operations
REST Clients	Integrate external APIs	REST client definitions with operations
OQL Queries	Build complex data retrieval	Optimized queries with joins
Security	Configure access control	Module roles, user roles, GRANT/REVOKE

1.5.2 Skill Combinations

Example 1: Complete Entity Module

Request: "Create a Product catalog with name, price, category, and inventory tracking"

AI combines: Domain Model + Overview Page + Detail Page + Validation

```
/**
 * === DOMAIN MODEL ===
 * Generated by: Domain Model Skill
 */
CREATE MODULE ProductCatalog;

/** Product categories for catalog organization */
CREATE ENUMERATION ProductCatalog.ProductCategory (
    Electronics CAPTION 'Electronics',
    Clothing CAPTION 'Clothing',
    HomeGarden CAPTION 'Home & Garden',
    Sports CAPTION 'Sports & Outdoors',
    Books CAPTION 'Books & Media'
);

/**
 * Product catalog with inventory tracking.
 * Central product master data for e-commerce operations.
 */
@Position(100, 100)
CREATE PERSISTENT ENTITY ProductCatalog.Product (
    /** Product identification */
    SKU: String(50) NOT NULL UNIQUE,
    Name: String(200) NOT NULL,
    Description: String(2000),
```

```
    /** Pricing */
    Price: Decimal NOT NULL,
    CostPrice: Decimal,

    /** Categorization */
    Category: Enum ProductCatalog.ProductCategory NOT NULL,

    /** Inventory */
    StockQuantity: Integer DEFAULT 0,
    ReorderLevel: Integer DEFAULT 10,
    IsActive: Boolean DEFAULT true,

    /** Audit fields */
    CreatedAt: DateTime DEFAULT '[%CurrentDateTime%]',
    ModifiedAt: DateTime
);

CREATE INDEX idx_Product_Category
    ON ProductCatalog.Product (Category, IsActive);

/**
 * === VALIDATION MICROFLOWS ===
 * Generated by: Validation Skill
 */
CREATE MICROFLOW ProductCatalog.ValidateProduct($product: ProductCatalog.Product)
RETURNS Boolean AS $isValid
FOLDER 'Validation'
BEGIN
    DECLARE $isValid Boolean = true;

    /** SKU format validation */
    IF length($product/SKU) < 3 THEN
        VALIDATION FEEDBACK $product/SKU 'SKU must be at least 3 characters';
        SET $isValid = false;
    END IF;

    /** Price validation */
    IF $product/Price <= 0 THEN
        VALIDATION FEEDBACK $product/Price 'Price must be greater than zero';
        SET $isValid = false;
    END IF;

    /** Margin validation */
    IF $product/CostPrice != empty AND $product/CostPrice >= $product/Price THEN
        VALIDATION FEEDBACK $product/CostPrice 'Cost price should be less than
selling price';
        SET $isValid = false;
    END IF;

    /** Inventory validation */
    IF $product/ReorderLevel < 0 THEN
        VALIDATION FEEDBACK $product/ReorderLevel 'Reorder level cannot be negative';
        SET $isValid = false;
    END IF;
```

```
    RETURN $isValid;
END;

/**
 * === OVERVIEW PAGE ===
 * Generated by: Overview Page Skill
 */
CREATE PAGE ProductCatalog.ProductOverview
(
  Title: 'Product Catalog',
  Layout: Atlas_Core.Atlas_Default,
  Url: 'products'
)
{
  LAYOUTGRID mainGrid {
    ROW row1 {
      COLUMN col1 (DesktopWidth: 12) {
        DYNAMICTEXT pageTitle (Content: 'Product Catalog', RenderMode: H3)
      }
    }
    ROW row2 {
      COLUMN col2 (DesktopWidth: 12) {
        DATAGRID ProductGrid (
          DataSource: DATABASE ProductCatalog.Product,
          OrderBy: Name ASC
        ) {
          CONTROLBAR bar1 {
            ACTIONBUTTON btnNew (
              Caption: 'New Product',
              Action: CREATE_OBJECT ProductCatalog.Product THEN SHOW_PAGE
ProductCatalog.Product_NewEdit,
              ButtonStyle: Primary
            )
          }
          COLUMN colSku (Attr: SKU, Caption: 'SKU')
          COLUMN colName (Attr: Name, Caption: 'Product Name')
          COLUMN colCat (Attr: Category, Caption: 'Category')
          COLUMN colPrice (Attr: Price, Caption: 'Price')
          COLUMN colStock (Attr: StockQuantity, Caption: 'Stock')
          COLUMN colActive (Attr: IsActive, Caption: 'Active')
          COLUMN colActions (Caption: 'Actions') {
            ACTIONBUTTON btnEdit (
              Caption: 'Edit',
              Action: SHOW_PAGE ProductCatalog.Product_NewEdit
            )
            ACTIONBUTTON btnDelete (Caption: 'Delete', Action: DELETE, ButtonStyle:
Danger)
          }
        }
      }
    }
  }
}
```

```
/**
 * === DETAIL PAGE ===
 * Generated by: Detail Page Skill
 *
 * @param $product The product to edit
 */
CREATE PAGE ProductCatalog.Product_NewEdit
(
  Params: { $product: ProductCatalog.Product },
  Title: 'Product Details',
  Layout: Atlas_Core.PopupLayout
)
{
  DATAVIEW ProductForm (DataSource: $product) {
    LAYOUTGRID innerGrid {
      ROW row1 {
        COLUMN col1 (DesktopWidth: 4) {
          TEXTBOX txtSKU (Label: 'SKU', Binds: SKU)
        }
        COLUMN col2 (DesktopWidth: 8) {
          TEXTBOX txtName (Label: 'Product Name', Binds: Name)
        }
      }
      ROW row2 {
        COLUMN col3 (DesktopWidth: 12) {
          TEXTAREA txtDescription (Label: 'Description', Binds: Description)
        }
      }
      ROW row3 {
        COLUMN col4 (DesktopWidth: 4) {
          TEXTBOX txtPrice (Label: 'Price', Binds: Price)
        }
        COLUMN col5 (DesktopWidth: 4) {
          TEXTBOX txtCostPrice (Label: 'Cost Price', Binds: CostPrice)
        }
        COLUMN col6 (DesktopWidth: 4) {
          COMBOBOX cmbCategory (Label: 'Category', Binds: Category)
        }
      }
      ROW row4 {
        COLUMN col7 (DesktopWidth: 4) {
          TEXTBOX txtStock (Label: 'Stock Quantity', Binds: StockQuantity)
        }
        COLUMN col8 (DesktopWidth: 4) {
          TEXTBOX txtReorder (Label: 'Reorder Level', Binds: ReorderLevel)
        }
        COLUMN col9 (DesktopWidth: 4) {
          CHECKBOX cbIsActive (Label: 'Active', Binds: IsActive)
        }
      }
    }

    FOOTER footer1 {
```

```
      ACTIONBUTTON btnSave (Caption: 'Save', Action: SAVE_CHANGES, ButtonStyle:
Success)
      ACTIONBUTTON btnCancel (Caption: 'Cancel', Action: CANCEL_CHANGES)
    }
  }
}
```

Example 2: Master-Detail Pattern

Request: "Create order management with line items"

AI combines: Domain Model + Master-Detail Page + Calculation Microflows

```
/**
 * Order line items for master-detail pattern.
 * Each line represents a product in an order.
 */
@Position(250, 100)
CREATE PERSISTENT ENTITY Sales.OrderLine (
    ProductName: String(200) NOT NULL,
    Quantity: Integer NOT NULL DEFAULT 1,
    UnitPrice: Decimal NOT NULL,
    LineTotal: Decimal NOT NULL
);

/**
 * Links order lines to their parent order.
 * Cascade delete ensures lines are removed with the order.
 */
CREATE ASSOCIATION Sales.OrderLine_Order
    FROM Sales.OrderLine
    TO Sales.Order
    TYPE REFERENCE
    DELETE_BEHAVIOR CASCADE;

/**
 * Master-Detail page for order editing
 *
 * @param $order The order to edit
 */
CREATE PAGE Sales.OrderEdit
(
  Params: { $order: Sales.Order },
  Title: 'Edit Order',
  Layout: Atlas_Core.Atlas_Default,
  Url: 'order-edit'
)
{
  LAYOUTGRID mainGrid {
    /** Master section: Order header */
    ROW headerRow {
      COLUMN col1 (DesktopWidth: 12) {
        DATAVIEW OrderHeader (DataSource: $order) {
          LAYOUTGRID headerGrid {
```

```
            ROW row1 {
              COLUMN colNum (DesktopWidth: 4) {
                TEXTBOX txtOrderNumber (Label: 'Order #', Binds: OrderNumber)
              }
              COLUMN colDate (DesktopWidth: 4) {
                DATEPICKER dpOrderDate (Label: 'Order Date', Binds: OrderDate)
              }
              COLUMN colCust (DesktopWidth: 4) {
                REFERENCESELECTOR refCustomer (Label: 'Customer', Binds:
Sales.Order_Customer)
              }
            }
          }
        }
      }
    }

    /** Detail section: Order lines gallery + selected line */
    ROW detailRow {
      COLUMN colLines (DesktopWidth: 8) {
        DYNAMICTEXT linesTitle (Content: 'Order Lines', RenderMode: H4)
        GALLERY OrderLines (
          DataSource: DATABASE Sales.OrderLine,
          Selection: Single
        ) {
          DYNAMICTEXT prodName (Content: 'Product', RenderMode: H4)
          DYNAMICTEXT qty (Content: 'Quantity')
          DYNAMICTEXT price (Content: 'Unit Price')
          DYNAMICTEXT total (Content: 'Line Total')
        }
        ACTIONBUTTON btnAddLine (
          Caption: 'Add Line',
          Action: CREATE_OBJECT Sales.OrderLine THEN SHOW_PAGE Sales.OrderLine_Edit,
          ButtonStyle: Primary
        )
      }

      /** Selected line detail */
      COLUMN colDetail (DesktopWidth: 4) {
        DYNAMICTEXT detailTitle (Content: 'Line Details', RenderMode: H4)
        DATAVIEW LineDetail (DataSource: SELECTION OrderLines) {
          TEXTBOX txtProdName (Label: 'Product', Binds: ProductName)
          TEXTBOX txtQty (Label: 'Quantity', Binds: Quantity)
          TEXTBOX txtUnitPrice (Label: 'Unit Price', Binds: UnitPrice)
          TEXTBOX txtLineTotal (Label: 'Line Total', Binds: LineTotal)

          FOOTER footer1 {
            ACTIONBUTTON btnRemove (Caption: 'Remove Line', Action: DELETE,
ButtonStyle: Danger)
          }
        }
      }
    }
```

```
    /** Order totals and actions */
    ROW actionsRow {
      COLUMN colTotal (DesktopWidth: 8) {
        DYNAMICTEXT orderTotal (Content: 'Order Total', RenderMode: H3)
      }
      COLUMN colActions (DesktopWidth: 4) {
        ACTIONBUTTON btnSave (Caption: 'Save', Action: SAVE_CHANGES, ButtonStyle:
Success)
        ACTIONBUTTON btnSubmit (Caption: 'Submit Order', Action: MICROFLOW
Sales.SubmitOrder, ButtonStyle: Primary)
        ACTIONBUTTON btnCancel (Caption: 'Cancel', Action: CANCEL_CHANGES)
      }
    }
  }
}
```

1.6 Defining Best Practices and Guardrails

Organizations can define MDL templates that encode their standards:

1.6.1 Entity Naming Convention Template

```
/**
 * Template: Standard Audit Fields.
 * All persistent entities should include these fields.
 * ${EntityDescription}
 */
@Position(${X}, ${Y})
CREATE PERSISTENT ENTITY ${Module}.${EntityName} (
    /** Business fields go here */
    ${BusinessFields}

    /** === STANDARD AUDIT FIELDS (Required) === */

    /** Record creation timestamp */
    CreatedAt: DateTime DEFAULT '[%CurrentDateTime%]',

    /** User who created this record */
    CreatedBy: String(200),

    /** Last modification timestamp */
    ModifiedAt: DateTime,

    /** User who last modified this record */
    ModifiedBy: String(200),

    /** Soft delete flag - never hard delete records */
    IsDeleted: Boolean DEFAULT false
);

/** Audit index for query performance */
```

```
CREATE INDEX idx_${EntityName}_audit
    ON ${Module}.${EntityName} (CreatedAt, ModifiedAt);
```

1.6.2 Microflow Error Handling Template

```
/**
 * Template: Microflow with Standard Error Handling
 * Use ON ERROR blocks on specific activities that may fail.
 */
CREATE MICROFLOW ${Module}.${MicroflowName}(
    ${Parameters}
) RETURNS ${ReturnType} AS $result
BEGIN
    /** Initialize result */
    DECLARE $result ${ReturnType};

    /** Activity with error handling */
    CALL MICROFLOW ${Module}.RiskyOperation()
    ON ERROR {
        /** Log error with context */
        LOG ERROR NODE 'ErrorHandling' 'Error in ${MicroflowName}';
        RETURN $result;
    };

    /** === BUSINESS LOGIC HERE === */
    ${BusinessLogic}

    RETURN $result;
END;
```

1.6.3 Page Layout Standards

```
/**
 * Template: Standard List Page
 * Use this template for all overview/list pages
 */
CREATE PAGE ${Module}.${Entity}Overview
(
  Title: '${EntityDisplayName} Overview',
  Layout: Atlas_Core.Atlas_Default,
  Url: '${entity-plural}'
)
{
  LAYOUTGRID mainGrid {
    ROW row1 {
      COLUMN col1 (DesktopWidth: 12) {
        DYNAMICTEXT pageTitle (Content: '${EntityDisplayName} Management',
RenderMode: H3)
      }
    }
    ROW row2 {
```

```
      COLUMN col2 (DesktopWidth: 12) {
        DATAGRID ${Entity}Grid (
          DataSource: DATABASE ${Module}.${Entity},
          OrderBy: Name ASC
        ) {
          CONTROLBAR bar1 {
            ACTIONBUTTON btnNew (
              Caption: 'New',
              Action: CREATE_OBJECT ${Module}.${Entity} THEN SHOW_PAGE ${Module}.
${Entity}_NewEdit,
              ButtonStyle: Primary
            )
          }
          /** Entity-specific columns */
          ${DataGridColumns}
          COLUMN colActions (Caption: 'Actions') {
            ACTIONBUTTON btnEdit (Caption: 'Edit', Action: SHOW_PAGE ${Module}.
${Entity}_NewEdit)
            ACTIONBUTTON btnView (Caption: 'View', Action: SHOW_PAGE ${Module}.
${Entity}_View)
            ACTIONBUTTON btnDelete (Caption: 'Delete', Action: DELETE, ButtonStyle:
Danger)
          }
        }
      }
    }
  }
}
```

1.7 Agentic Search: AI-Powered Code Understanding

mxcli provides powerful search capabilities that enable AI assistants to understand and navigate large Mendix projects:

1.7.1 Catalog Queries

The built-in catalog provides SQL access to project metadata:

```
/** Find all entities in a module */
SELECT Name, EntityType, Description
FROM CATALOG.ENTITIES
WHERE ModuleName = 'Sales';

/** Find microflows that reference a specific entity */
SELECT Name, ModuleName
FROM CATALOG.MICROFLOWS
WHERE Description LIKE '%Customer%'
   OR Name LIKE '%Customer%';

/** Get entity relationships (associations) */
SELECT Name, ParentEntity, ChildEntity, AssociationType
```

```sql
FROM CATALOG.ASSOCIATIONS
WHERE ParentEntity LIKE '%Order%'
   OR ChildEntity LIKE '%Order%';

/** Find pages by URL pattern */
SELECT Name, ModuleName, URL
FROM CATALOG.PAGES
WHERE URL LIKE '/admin%';
```

1.7.2 Advanced Catalog Queries

1.7.2.1 Duplicate Entity Detection

```sql
/**
 * Find entities with similar names across modules
 * Helps identify potential consolidation opportunities
 */
SELECT
    e1.ModuleName AS Module1,
    e1.Name AS Entity1,
    e2.ModuleName AS Module2,
    e2.Name AS Entity2
FROM CATALOG.ENTITIES e1
JOIN CATALOG.ENTITIES e2
    ON LOWER(e1.Name) = LOWER(e2.Name)
    AND e1.ModuleName < e2.ModuleName
ORDER BY e1.Name;

/**
 * Find entities with identical attribute patterns
 * Detects copy-paste inheritance anti-patterns
 */
SELECT
    e1.QualifiedName AS Entity1,
    e2.QualifiedName AS Entity2,
    COUNT(*) AS MatchingAttributes
FROM CATALOG.ATTRIBUTES a1
JOIN CATALOG.ATTRIBUTES a2
    ON a1.Name = a2.Name
    AND a1.AttributeType = a2.AttributeType
    AND a1.EntityId != a2.EntityId
JOIN CATALOG.ENTITIES e1 ON a1.EntityId = e1.Id
JOIN CATALOG.ENTITIES e2 ON a2.EntityId = e2.Id
WHERE e1.QualifiedName < e2.QualifiedName
GROUP BY e1.QualifiedName, e2.QualifiedName
HAVING COUNT(*) >= 3
ORDER BY MatchingAttributes DESC;

/**
 * Find potential enum candidates
 * Entities with only String attributes and few instances
 */
SELECT
```

```sql
    e.QualifiedName,
    COUNT(a.Id) AS AttributeCount
FROM CATALOG.ENTITIES e
JOIN CATALOG.ATTRIBUTES a ON e.Id = a.EntityId
WHERE a.AttributeType = 'String'
GROUP BY e.QualifiedName
HAVING COUNT(a.Id) <= 2
   AND COUNT(a.Id) = (
       SELECT COUNT(*) FROM CATALOG.ATTRIBUTES
       WHERE EntityId = e.Id
   );
```

1.7.2.2 Data Lineage Analysis

```sql
/**
 * Entity dependency graph via associations
 * Find all entities connected to a given entity
 */
SELECT
    'Outgoing' AS Direction,
    a.Name AS Association,
    a.ChildEntity AS RelatedEntity,
    a.AssociationType AS Type
FROM CATALOG.ASSOCIATIONS a
WHERE a.ParentEntity = 'Sales.Order'
UNION ALL
SELECT
    'Incoming' AS Direction,
    a.Name AS Association,
    a.ParentEntity AS RelatedEntity,
    a.AssociationType AS Type
FROM CATALOG.ASSOCIATIONS a
WHERE a.ChildEntity = 'Sales.Order';

/**
 * Find microflows that modify an entity
 * Track all business logic touching specific data
 */
SELECT
    m.QualifiedName AS Microflow,
    m.ReturnType,
    m.Description
FROM CATALOG.MICROFLOWS m
WHERE m.ObjectUsage LIKE '%Sales.Customer%'
   OR m.Parameters LIKE '%Sales.Customer%'
   OR m.ReturnType LIKE '%Sales.Customer%';

/**
 * Page-to-entity mapping
 * Which pages display or edit which entities
 */
SELECT
    p.QualifiedName AS Page,
```

```
    p.DataSource AS Entity,
    p.WidgetTypes
FROM CATALOG.PAGES p
WHERE p.DataSource IS NOT NULL
ORDER BY p.DataSource, p.QualifiedName;

/**
 * Complete data flow for an entity
 * Pages → Entity → Microflows chain
 */
SELECT 'Page' AS Source, p.QualifiedName AS Element, 'displays' AS Action
FROM CATALOG.PAGES p
WHERE p.DataSource LIKE '%Customer%'
UNION ALL
SELECT 'Microflow' AS Source, m.QualifiedName AS Element, 'processes' AS Action
FROM CATALOG.MICROFLOWS m
WHERE m.ObjectUsage LIKE '%Customer%';
```

1.7.2.3 Quality Analysis Queries

```
/**
 * Entities without documentation
 * Identify technical debt in documentation
 */
SELECT
    QualifiedName,
    EntityType,
    AttributeCount
FROM CATALOG.ENTITIES
WHERE (Description IS NULL OR Description = '')
ORDER BY AttributeCount DESC;

/**
 * Documentation coverage by module
 */
SELECT
    ModuleName,
    COUNT(*) AS TotalEntities,
    SUM(CASE WHEN Description != '' THEN 1 ELSE 0 END) AS Documented,
    ROUND(100.0 * SUM(CASE WHEN Description != '' THEN 1 ELSE 0 END) / COUNT(*), 1)
AS CoveragePercent
FROM CATALOG.ENTITIES
GROUP BY ModuleName
ORDER BY CoveragePercent ASC;

/**
 * Complex entities (many attributes/associations)
 * Candidates for refactoring
 */
SELECT
    e.QualifiedName,
    e.AttributeCount,
    COUNT(a.Id) AS AssociationCount,
```

```sql
    e.AttributeCount + COUNT(a.Id) AS ComplexityScore
FROM CATALOG.ENTITIES e
LEFT JOIN CATALOG.ASSOCIATIONS a
    ON e.QualifiedName = a.ParentEntity
    OR e.QualifiedName = a.ChildEntity
GROUP BY e.QualifiedName, e.AttributeCount
HAVING ComplexityScore > 15
ORDER BY ComplexityScore DESC;

/**
 * Orphan detection - entities with no associations
 * May indicate missing relationships
 */
SELECT e.QualifiedName, e.EntityType, e.AttributeCount
FROM CATALOG.ENTITIES e
LEFT JOIN CATALOG.ASSOCIATIONS a
    ON e.QualifiedName = a.ParentEntity
    OR e.QualifiedName = a.ChildEntity
WHERE a.Id IS NULL
  AND e.EntityType = 'Persistent'
ORDER BY e.ModuleName, e.Name;

/**
 * Naming convention violations
 * Find entities not following PascalCase
 */
SELECT QualifiedName, Name
FROM CATALOG.ENTITIES
WHERE Name != UPPER(SUBSTR(Name, 1, 1)) || SUBSTR(Name, 2)
   OR Name LIKE '%_%'
   OR Name LIKE '% %';

/**
 * Module health dashboard
 * Overview of module complexity and documentation
 */
SELECT
    ModuleName,
    COUNT(DISTINCT e.Id) AS Entities,
    COUNT(DISTINCT m.Id) AS Microflows,
    COUNT(DISTINCT p.Id) AS Pages,
    ROUND(100.0 * SUM(CASE WHEN e.Description != '' THEN 1 ELSE 0 END) /
        NULLIF(COUNT(DISTINCT e.Id), 0), 1) AS DocCoverage
FROM CATALOG.ENTITIES e
LEFT JOIN CATALOG.MICROFLOWS m ON e.ModuleName = m.ModuleName
LEFT JOIN CATALOG.PAGES p ON e.ModuleName = p.ModuleName
GROUP BY ModuleName
ORDER BY Entities DESC;
```

1.7.2.4 Security & Access Analysis

For hands-on security management (creating roles, granting access), see the Security Management section above.

The catalog also enables SQL-based security analysis:

```
/**
 * Entities with no access rules defined
 * Potential security concern
 */
SELECT
    e.QualifiedName,
    e.EntityType,
    e.AccessRuleCount
FROM CATALOG.ENTITIES e
WHERE e.AccessRuleCount = 0
  AND e.EntityType = 'Persistent'
ORDER BY e.ModuleName;

/**
 * Wide-open entities (all attributes readable by all roles)
 * Review for data sensitivity
 */
SELECT
    e.QualifiedName,
    e.AttributeCount,
    ar.UserRole,
    ar.AllowRead,
    ar.AllowWrite
FROM CATALOG.ENTITIES e
JOIN CATALOG.ACCESS_RULES ar ON e.Id = ar.EntityId
WHERE ar.AllowRead = 1
  AND ar.UserRole = 'Anonymous'
ORDER BY e.AttributeCount DESC;
```

1.7.3 Interactive Discovery

```
/** List all modules in the project */
SHOW MODULES;

/** Explore entities in a module */
SHOW ENTITIES IN Sales;

/** Get the MDL definition of any element */
DESCRIBE ENTITY Sales.Customer;
DESCRIBE MICROFLOW Sales.SubmitOrder;
DESCRIBE PAGE Sales.CustomerOverview;

/** View associations */
SHOW ASSOCIATIONS IN Sales;
```

1.7.4 How AI Assistants Use This

When you ask Claude Code to “add a discount field to orders”, it:

1. **Searches** the catalog to find the Order entity
2. **Analyzes** existing patterns in your project
3. **Generates** MDL that matches your conventions
4. **Validates** the change won’t break existing logic

5. **Applies** the change and shows you the diff

```
/** AI discovers the entity */
DESCRIBE ENTITY Sales.Order;

/**
 * AI generates the modification.
 * Adds discount percentage applied to order (0-100).
 */
ALTER ENTITY Sales.Order
    ADD ATTRIBUTE DiscountPercent: Decimal DEFAULT 0;

/**
 * AI updates the total calculation microflow
 * (based on analysis of existing patterns)
 */
```

1.8 Future Vision

1.8.1 REST Service Integration

```
/** Define a REST service consumed by your app */
CREATE REST CLIENT PaymentGateway.StripeAPI
BASE URL 'https://api.stripe.com/v1'
AUTHENTICATION BASIC
    USERNAME ''
    PASSWORD '[%StripeSecretKey%]'
TIMEOUT 30000
BEGIN
    OPERATION CreateCharge
        METHOD POST
        PATH '/charges'
        BODY '{
            "amount": ${amount},
            "currency": "${currency}",
            "source": "${token}",
            "description": "${description}"
        }'
        PARAMETER amount: Integer IN BODY
        PARAMETER currency: String IN BODY
        PARAMETER token: String IN BODY
        PARAMETER description: String IN BODY
        RESPONSE 200 PaymentGateway.ChargeResponse;

    OPERATION GetCharge
        METHOD GET
        PATH '/charges/${chargeId}'
        PARAMETER chargeId: String IN PATH
        RESPONSE 200 PaymentGateway.ChargeResponse;
END;
```

1.8.2 Database Connector

```
/** Connect to external databases */
CREATE DATABASE CONNECTION Warehouse.LegacyERP
    TYPE 'postgresql'
    HOST 'erp.internal.company.com'
    PORT 5432
    DATABASE 'erp_production'
    USERNAME '[%ERPUsername%]'
    PASSWORD '[%ERPPassword%]';

/** Define queries as reusable operations */
CREATE DATABASE QUERY Warehouse.GetInventoryLevels
    CONNECTION Warehouse.LegacyERP
    SQL 'SELECT product_code, quantity, warehouse_id
         FROM inventory
         WHERE quantity < $1'
    PARAMETER threshold: Integer
    RETURNS List of Warehouse.InventoryLevel
    MAP (
        ProductCode = product_code,
        Quantity = quantity,
        WarehouseId = warehouse_id
    );
```

1.8.3 OData Service Publishing

```
/** Publish entities as OData services */
CREATE ODATA SERVICE Sales.OrderAPI
VERSION '4.0'
PATH '/odata/sales'
AUTHENTICATION 'MendixSSO'
BEGIN
    PUBLISH ENTITY Sales.Order AS 'Orders'
        READABLE true
        INSERTABLE true
        UPDATABLE true
        DELETABLE false
        EXPAND Customer, OrderLines;

    PUBLISH ENTITY Sales.Customer AS 'Customers'
        READABLE true
        INSERTABLE false
        FILTER BY Name, Email, IsActive;
END;
```

1.8.4 Workflow Integration

```
/** Define business workflows */
CREATE WORKFLOW Sales.OrderApproval
FOR Sales.Order
BEGIN
```

```
    STATE Draft
        ON Submit GOTO PendingApproval
            WHEN TotalAmount > 0;

    STATE PendingApproval
        ASSIGN TO Role 'Sales.Manager'
        ON Approve GOTO Approved
        ON Reject GOTO Rejected
            REQUIRE Comment;

    STATE Approved
        ON Ship GOTO Shipped
        TRIGGER MICROFLOW Sales.NotifyCustomer;

    STATE Shipped
        ON Deliver GOTO Completed;

    STATE Completed
        FINAL;

    STATE Rejected
        ON Revise GOTO Draft
        FINAL;
END;
```

1.8.5 VS Code Extension and Language Server

The `vscode-mdl` extension provides comprehensive MDL language support in VS Code, Cursor, and other compatible editors. The extension is **auto-installed** when you run `mxcli init`.

1.8.5.1 Installation

Automatic (Recommended):

```
# Initialize project - auto-installs extension if `code` CLI is available
mxcli init /path/to/mendix-project
```

Manual Installation:

```
# Build extension package
make vscode-ext           # Builds .vsix package

# Install into VS Code
make vscode-install       # Builds and installs

# Or install from .vsix manually
code --install-extension vscode-mdl/vscode-mdl-*.vsix
```

If `code` CLI isn't available during `mxcli init`, the extension is extracted to `.claude/vscode-mdl.vsix` for manual installation.

1.8.5.2 Features

Syntax and Validation: - **Syntax Highlighting** - Color-coded MDL keywords, types, strings, comments (TextMate grammar) - **Parse Diagnostics** - Real-time error detection as you type (no project required) - **Semantic Diagnostics** - Reference validation on save (checks entity/microflow/page references against `.mpr`) - **Document Symbols** - Outline view listing all CREATE/ALTER/DROP statements - **Folding Ranges** - Collapsible BEGIN/END, IF/END IF, LOOP/END LOOP, braces, parentheses

Navigation: - **Hover** - Shows MDL definition for `Module.Entity` qualified names (via `mxcli describe`) - **Go to Definition** - Ctrl+click/Cmd+click on qualified names opens virtual MDL document - **Code Completion** - Context-aware keyword suggestions (e.g., after `CREATE`, `SHOW`)

Execution: - **Context Menu Commands:** - **Run File** - Execute entire MDL file against project - **Run Selection** - Execute selected MDL statements - **Check File** - Validate syntax and references without execution

Example - Right-click context menu:

```
┌──────────────────────────────────────┐
│  your-script.mdl                     │
│  ┌──────────────────────────────────┐│
│  │ CREATE PERSISTENT ENTITY ...     │
│  │                                  │
│  │ [Right-click menu]               │
│  │ ├─ Run File                      │
│  │ ├─ Run Selection                 │
│  │ ├─ Check File                    │
│  │ └─ Check File (with references)  │
└──┴──────────────────────────────────┘
```

1.8.5.3 Configuration

VS Code Settings (`settings.json`):

```
{
  // Path to mxcli binary (default: "mxcli" from PATH)
  "mdl.mxcliPath": "/path/to/mxcli",

  // Path to .mpr file (auto-discovered if empty)
  "mdl.mprPath": "/path/to/project.mpr"
}
```

Project Auto-Discovery: When `mdl.mprPath` is not set, the extension searches the workspace for `.mpr` files. If exactly one is found, it's used automatically.

Project-Aware Features: Features requiring project access (hover, definition, semantic diagnostics) gracefully degrade when no `.mpr` is configured. Core features (syntax highlighting, parse diagnostics, completion, symbols, folding) work without a project.

1.8.5.4 Language Server Protocol (LSP)

The extension communicates with `mxcli lsp --stdio` as the language server. This architecture enables: - Use with any LSP-compatible editor (not just VS Code) - Consistent behavior across editors - Single implementation for all MDL tooling

LSP Server Capabilities:

```
┌──────────────────────────────────────────────────────────────┐
│  VS Code / Cursor / Other LSP-Compatible Editors             │
├──────────────────────────────────────────────────────────────┤
│  your-script.mdl                                             │
│  ┌──────────────────────────────────────────────────────────┐│
│  │ CREATE PERSISTENT ENTITY Sales.                          │
│  │                           ┌──────────────────┐           │
│  │                           │ Customer         │           │
│  │                           │ Order            │           │
│  │                           │ Product          │           │
│  │                           │ OrderLine        │           │
│  │                           └──────────────────┘           │
│  │                           ▲ Autocomplete suggestions     │
│  └──────────────────────────────────────────────────────────┘│
└──────────────────────────────────────────────────────────────┘
```

Using LSP with Other Editors:

```
# Start language server manually (reads from stdin, writes to stdout)
mxcli lsp --stdio

# Configure your editor to spawn this command when opening .mdl files
```

1.8.5.5 Building from Source

If you're developing the extension or want the latest version:

```
# Build extension package
cd vscode-mdl
npm install
npm run compile
vsce package

# Install
code --install-extension vscode-mdl-*.vsix
```

Or use the Makefile shortcuts:

```
make vscode-ext        # Build .vsix package
make vscode-install    # Build and install
make sync-vsix         # Sync to cmd/mxcli for embedding
```

1.9 CLI Commands Reference

mxcli provides a rich set of commands for project analysis, code search, and development workflows.

1.9.1 Project Initialization

```
# Initialize with Claude Code (default)
mxcli init /path/to/mendix-project

# Initialize for a specific AI tool
mxcli init --tool cursor /path/to/mendix-project

# Initialize for multiple tools
mxcli init --tool claude --tool cursor --tool continue

# Initialize for all supported tools
mxcli init --all-tools

# List supported tools
mxcli init --list-tools

# What it creates (universal):
# - .ai-context/skills/ - MDL pattern guides (shared by all tools)
# - PROJECT_AI.md - Universal AI assistant guide
#
# What it creates (Claude Code specific):
# - .claude/settings.json - Claude Code permissions
# - .claude/commands/ - Slash commands
# - .claude/lint-rules/ - Starlark lint rules
# - CLAUDE.md - Project instructions
# - VS Code MDL extension (auto-installed if `code` CLI available)
```

1.9.2 Script Execution

```
# Execute an MDL script file
mxcli exec script.mdl -p app.mpr

# Execute commands directly
mxcli -p app.mpr -c "SHOW ENTITIES"
mxcli -p app.mpr -c "SHOW MODULES"

# Check syntax without executing (validation only)
mxcli check script.mdl

# Check syntax with reference validation (requires project)
mxcli check script.mdl -p app.mpr --references
```

1.9.3 Code Search and Navigation

These commands require a catalog refresh first (`REFRESH CATALOG FULL` or `mxcli -p app.mpr -c "REFRESH CATALOG FULL"`):

```
# Find what calls a microflow (callers)
mxcli callers -p app.mpr Module.MyMicroflow
mxcli callers -p app.mpr Module.MyMicroflow --transitive
```

```
# Find what a microflow calls (callees)
mxcli callees -p app.mpr Module.MyMicroflow

# Find all references to an element
mxcli refs -p app.mpr Module.Customer

# Analyze impact of changing an element
mxcli impact -p app.mpr Module.Customer

# Assemble context for LLM consumption
mxcli context -p app.mpr Module.MyMicroflow --depth 3

# Full-text search across strings and source
mxcli search -p app.mpr "validation"
mxcli search -p app.mpr "validation" -q --format names
mxcli search -p app.mpr "validation" -q --format json
```

Search Output Formats: - `names` (default) - `type<TAB>name` per line, suitable for piping - `json` - JSON array of results - `-q` flag - Quiet mode (no headers/formatting)

Example: Pipe search results to describe:

```
mxcli search -p app.mpr "error" -q --format names | head -1 | awk '{print $2}' | \
  xargs mxcli describe -p app.mpr microflow
```

1.9.4 Project Introspection

```
# Describe a specific element (outputs MDL source)
mxcli describe -p app.mpr entity Module.EntityName
mxcli describe -p app.mpr microflow Module.MicroflowName
mxcli describe -p app.mpr page Module.PageName
mxcli describe -p app.mpr snippet Module.SnippetName

# Show project elements
mxcli show -p app.mpr modules
mxcli show -p app.mpr entities --module MyModule
mxcli show -p app.mpr microflows --module MyModule
mxcli show -p app.mpr pages --module MyModule

# Output project structure as JSON
mxcli project-tree -p app.mpr
```

1.9.5 Linting

```
# Lint a project for issues
mxcli lint -p app.mpr

# With colored output
mxcli lint -p app.mpr --color

# JSON output for CI integration
```

```
mxcli lint -p app.mpr --format json

# SARIF output for GitHub integration
mxcli lint -p app.mpr --format sarif > results.sarif

# List available rules
mxcli lint -p app.mpr --list-rules

# Exclude modules from linting
mxcli lint -p app.mpr --exclude System --exclude Administration
```

Built-in Lint Rules: - **MDL001 (NamingConvention)** - PascalCase for entities, microflows, pages, enumerations - **MDL002 (EmptyMicroflow)** - No microflows with zero activities - **MDL003 (DomainModelSize)** - Max 15 persistent entities per domain model - **MDL004 (ValidationFeedback)** - Validation feedback must have non-empty message

1.9.6 Diff and Comparison

```
# Compare MDL script against project state (dry run preview)
mxcli diff -p app.mpr changes.mdl

# Compare local changes against git (MPR v2 only)
mxcli diff-local -p app.mpr --ref HEAD
mxcli diff-local -p app.mpr --ref HEAD~1
```

1.9.7 Styling Commands

```
# Discover available design properties from theme
mxcli -p app.mpr -c "SHOW DESIGN PROPERTIES"
mxcli -p app.mpr -c "SHOW DESIGN PROPERTIES FOR CONTAINER"

# Inspect styling on existing pages/snippets
mxcli -p app.mpr -c "DESCRIBE STYLING ON PAGE MyModule.CustomerEdit"
mxcli -p app.mpr -c "DESCRIBE STYLING ON PAGE MyModule.CustomerEdit WIDGET btnSave"

# Modify styling on existing pages/snippets
mxcli -p app.mpr -c "ALTER STYLING ON PAGE MyModule.CustomerEdit WIDGET btnSave SET
Class 'btn-lg'"
mxcli -p app.mpr -c "ALTER STYLING ON SNIPPET MyModule.NavMenu WIDGET ctn1 SET
'Spacing top' 'Large'"
```

1.9.8 Widget Discovery (EXPERIMENTAL)

Warning: These commands are proof-of-concept. Always use `DRY RUN` first and backup your project.

```
# Discover widgets in project (requires REFRESH CATALOG FULL)
./bin/mxcli -p app.mpr -c "SHOW WIDGETS"
./bin/mxcli -p app.mpr -c "SHOW WIDGETS IN MyModule"
./bin/mxcli -p app.mpr -c "SHOW WIDGETS WHERE WidgetType LIKE '%combobox%'"
```

```
# Preview changes (dry run)
./bin/mxcli -p app.mpr -c "UPDATE WIDGETS SET 'showLabel' = false WHERE WidgetType
LIKE '%combobox%' DRY RUN"

# Apply changes
./bin/mxcli -p app.mpr -c "UPDATE WIDGETS SET 'showLabel' = false WHERE WidgetType
LIKE '%combobox%' IN MyModule"
```

1.9.9 Language Server Protocol (LSP)

```
# Start LSP server for IDE integration
mxcli lsp --stdio

# VS Code extension uses this command automatically
# Configured via mdl.mxcliPath setting (default: "mxcli")
```

1.9.10 Catalog Management

The catalog is a SQLite database that caches project metadata for fast querying:

```
# Refresh catalog (within MDL commands)
mxcli -p app.mpr -c "REFRESH CATALOG"           # Quick refresh
mxcli -p app.mpr -c "REFRESH CATALOG FULL"      # Full rebuild with refs
mxcli -p app.mpr -c "REFRESH CATALOG FULL FORCE" # Force rebuild

# Query catalog tables
mxcli -p app.mpr -c "SELECT Name FROM CATALOG.MICROFLOWS LIMIT 10"
mxcli -p app.mpr -c "SHOW CATALOG TABLES"
```

Catalog is cached in `.mxcli/catalog.db` next to the MPR file.

1.9.11 Debug and Advanced Tools

```
# Dump raw BSON data for debugging widget/page issues
mxcli dump-bson -p app.mpr --doc "Module.PageName"

# Extract widget templates from a project
mxcli extract-templates -p app.mpr --output ./templates/

# Start HTTP visualization server
mxcli serve -p app.mpr --port 8080
```

1.9.12 Interactive REPL

```
# Start interactive REPL
mxcli

# Within REPL:
mendix> CONNECT LOCAL 'app.mpr';
mendix> SHOW MODULES;
```

```
mendix> SHOW ENTITIES IN MyModule;
mendix> DESCRIBE ENTITY MyModule.Customer;
```

1.10 Getting Started

1.10.1 1. Initialize Your Project

```
# Navigate to your Mendix project folder
cd /path/to/your-mendix-project

# Initialize mxcli (creates .ai-context/, CLAUDE.md, etc.)
./mxcli init
```

1.10.2 2. Connect and Explore

```
# Start the interactive REPL
./mxcli

# Connect to your project
mendix> CONNECT LOCAL 'MyProject.mpr';

# Explore the model
mendix> SHOW MODULES;
mendix> SHOW ENTITIES IN MyFirstModule;
mendix> DESCRIBE ENTITY MyFirstModule.Customer;
```

1.10.3 3. Use with Claude Code

Open your project in VS Code with Claude Code extension. The AI can now:

- Answer questions about your model
- Generate new entities, microflows, and pages
- Refactor existing elements
- Apply your organization's templates

1.10.4 4. Execute MDL Scripts

```
# Execute a script file
./mxcli -d "EXECUTE SCRIPT 'mdlsource/setup-sales-module.mdl'"

# Or run individual commands
./mxcli -d "SHOW MODULES"
./mxcli -d "CREATE MODULE Sales"
```

1.11 Summary

mxcli transforms Mendix development by providing:

Capability	Benefit
MDL Language	Human and AI-readable model definitions
Multi-Tool GenAI Integration	Works with Claude Code, Cursor, Continue.dev, Windsurf, Aider
Template System	Encode and enforce organizational standards
Agentic Search	AI-powered project understanding
Token Efficiency	5-10x fewer tokens than JSON, reducing costs and improving context
High Information Density	Complete microflow logic visible without clicking through activities

Whether you're a developer looking to accelerate your workflow or an organization wanting to leverage AI for Mendix development, mxcli provides the foundation for the next generation of low-code development.

1.12 Resources

- **Documentation**: docs/
- **MDL Syntax Reference**: docs/02-features/mdl/
- **Skills for AI Assistants**: .ai-context/skills/
- **Example Projects**: templates/

---

mxcli - Bringing the power of AI to Mendix development